形式手法による**AI**エージェント駆動開発 — マルチエージェント協調時代における人間の役割

安藤光太郎 — *IID Systems*

— 形式仕様を「契約」とした*AI*エージェント群がシステムを自律構築する近未来 —

はじめに：**AI**エージェント駆動開発の時代へ

大規模言語モデル（LLM）の急速な進化により、ソフトウェア開発のあり方が根本的に問い直されている。GitHub Copilot、Claude、GPT-4をはじめとするAIは、すでにコード生成・レビュー・リファクタリングにおいて実用的な能力を示している。しかし、この流れが向かう先は「AIをツールとして使う」という段階を超え、複数の**AI**エージェントがチーム開発のように協調し、システムを自律的に構築するという新たなパラダイムだ。

ここで根本的な問いが生じる。複数のAIエージェントが協調するとき、何が「チームの共通言語」になるのか？　自然言語で書かれた曖昧な仕様書では、エージェント間の解釈の齟齬を排除できない。テスト駆動開発（TDD）を適用しても、テストは帰納的推論に基づいており、エージェント間の仕様合意メカニズムとしては機能しない。

本稿が提案するのは、VDM（Vienna Development Method）をはじめとする形式手法を仕様記述の中核に据え、形式仕様をエージェント間の厳密な契約として機能させるマルチエージェント開発パラダイムである。建築に喩えれば、施主（人間）は「日当たりが良くて開放的なリビングが欲しい」という要望を伝え、ハウスメーカー（AI）がその要望と予算・立地条件・気候などを踏まえて具体的な設計を提案する。基礎工事・躯体・電気・配管をどの順序で施工するかは施主の関心事ではなく、ハウスメーカーが決定する。各工程の施工チーム（AIエージェント）は形式仕様という「設計図面」によって互いの境界を厳密に把握し、曖昧性のない協調を実現する。

本稿ではまず、TDDの本質的限界を指摘した上で（第1章）、形式手法がなぜAIエージェント間の「契約」として理想的に機能するかを論じ（第2-5章）、マルチエージェント協調アーキテクチャの具体像を示す（第7.3節）。そして、この新しいパラダイムにおいて人間の役割がどう再定義されるか——ドメインエキスパートとして要件を定義し、技術的な設計・実装はAIに委ねる——を議論する。

第1章：テスト駆動開発はなぜナンセンスか

なお、本稿で批判するのは「テスト駆動開発（TDD）というパラダイム」——テストを設計と品質保証の中心に据える思想——であり、テストという行為そのものではない。テストが補完的な役割で存続する点については第7.2節で詳述する。

1.1 テストの本質的限界——帰納的推論の罠

TDDの根本的な問題は、テストが帰納的推論に基づいている点にある。

テストとは「有限個の入力に対して期待する出力が得られるか」を検証する行為だ。しかし、ほとんどのプログラムの入力空間は無限、あるいは天文学的に巨大である。Edsger W. Dijkstraの有名な言葉を引用しよう。

> *“Program testing can be used to show the presence of bugs, but never to show their absence.”*（プログラムのテストはバグの存在を示すことができるが、バグの不在を示すことは決してできない。）*— Edsger W. Dijkstra, 1969*

これは単なる格言ではなく、論理学的に厳密な指摘である。テストケース $\{t_1, t_2, ..., t_n\}$ がすべて通過したとしても、テストされていない入力 t_{n+1} でプログラムが正しく動作する保証はない。これは不完全帰納法の問題そのものだ。

1.2 具体的な数理的議論

整数の加算関数 `add(a, b)` を考える。仮に `a` と `b` がそれぞれ32ビット整数だとすると、入力の組み合わせは $2^{32} \times 2^{32} = 2^{64} \approx 1.8 \times 10^{19}$ 通りである。仮に1テストケースを1ナノ秒で実行できたとしても、全網羅には約585年かかる。

実際のシステムでは、入力は整数ペアどころではない。APIリクエスト、データベース状態、タイミング、並行処理の順序など、状態空間は事実上無限である。TDDで書くテストは、この広大な空間のごくわずかな点を検査しているに過ぎない。

1.3 TDDの「設計手法としての価値」への反論

TDD推進者はしばしば「TDDはテスト手法ではなく設計手法だ」と主張する。テストを先に書くことで、インターフェースが使いやすくなる、という論理だ。

しかし、この議論にも問題がある。設計の良し悪しを評価する基準がテストの書きやすさに還元されてしまうからだ。テストしやすいインターフェースが必ずしも良いインターフェースとは限らない。本来、設計は要件の論理的構造から導出されるべきであり、テストの都合から逆算されるべきではない。

形式手法では、仕様そのものが設計を駆動する。不変条件（invariant）、事前条件（pre-condition）、事後条件（post-condition）を明示的に記述することで、インターフェースの正しさが仕様レベルで保証される。テストケースの偶然的な選択に設計を委ねる必要はない。

1.4 カバレッジという幻想

コードカバレッジ100%は、しばしば品質の指標として引用される。しかし、カバレッジが測定しているのは「コードのどの行が実行されたか」であって「仕様のどの部分が検証されたか」ではない。

以下の関数を考えてみよう。

```
def divide(a: int, b: int) -> float:
    return a / b
```

テスト `divide(10, 2) == 5.0` を書けば行カバレッジは100%になる。しかし `b = 0` のケースは検証されていない。カバレッジは「テストの網羅性」を測る指標としては根本的に不十分だ。

一方、形式仕様では次のように記述する。

```
divide(a: int, b: int) -> float
  pre: b ≠ 0
  post: result * b = a
```

事前条件 `b ≠ 0` が明示されることで、ゼロ除算の問題は仕様レベルで解決される。テストケースの漏れに依存しない。

第2章：形式手法とは何か——VDMを中心に

2.1 形式手法の基本的な考え方

形式手法（Formal Methods）とは、数学的な表記法と推論規則を使ってソフトウェアの仕様を記述し、その正しさを証明・検証する手法の総称だ。

主な形式手法には以下がある。

- **VDM**（**Vienna Development Method**）：1970年代にIBMウィーン研究所で開発。モデル指向の仕様記述言語VDM-SLを持つ。産業界での適用実績が豊富。
- **Z**記法：オックスフォード大学発。集合論とスキーマに基づく仕様記述。
- **B Method**：パリ地下鉄の自動運転システムなどで使用。自動証明との親和性が高い。
- **Alloy**：MIT発の軽量形式手法。モデル検査による自動検証が可能。
- **TLA+**：Leslie Lamportが開発。分散システムの仕様記述に特に強い。Amazonが大規模に採用。

本稿ではVDMを中心に据える。その理由は、VDM-SLはAIが生成・操作しやすい表記法であること、Overture Tool等による機械的な整合性検証との親和性が高いこと、そして産業界での適用実績が豊富であることだ。

2.2 VDM-SLの基本要素

VDM-SL（VDM Specification Language）の基本的な構成要素を簡潔に紹介する。

型定義（**Type Definitions**）：　VDM-SLでは、システムが扱うデータの構造を型として定義する。基本型（`nat`、`int`、`bool`、`char`など）に加え、集合型（`set of T`）、列型（`seq of T`）、写像型（`map T1 to T2`）、直積型、合併型などを組み合わせて複雑なデータ構造を表現する。

不変条件（**Invariants**）：　型や状態が常に満たすべき条件を述語論理で記述する。これにより、不正な状態がそもそも存在し得ないことを仕様レベルで保証する。

操作（**Operations**）：　状態を変更する操作を、事前条件（pre）と事後条件（post）のペアで定義する。「この操作を呼ぶ前に何が成り立っていなければならないか」「この操作が完了した後に何が成り立つか」を明示する。

関数（**Functions**）：　副作用のない純粋な計算を定義する。陰仕様（implicit specification）では、入出力の関係を述語で記述するだけで、具体的なアルゴリズムは指定しない。

2.3 演繹的推論 vs 帰納的推論

TDDと形式手法の決定的な違いは、推論の方向にある。

TDD（帰納的）は「具体的なテストケースの集合から、プログラムの一般的な正しさを推測する」。一方、形式仕様記述（演繹的）は「仕様を厳密に定義することで、実装の正しさの検証基準を明確化する」。

ここで重要な区別がある。形式仕様記述と形式検証は別のプロセスだ。VDM-SLで仕様を書くこと自体は、仕様の内部一貫性（矛盾がないこと）と完全性（必要な操作が定義されていること）を保証する。一方、実装が仕様に適合することの検証は、定理証明器による形式検証やプロパティベーステストなど別の手段で行う。本稿が提案するAI駆動開発では、後者を第4章Phase 4で扱う。

それでも、帰納的推論はどれだけ事例を積み重ねても確実性には到達しないという根本的限界がある。形式仕様に基づくアプローチは、少なくとも「何が正しいか」の基準を曖昧さなく定義し、その基準に照らした検証を可能にする点で、TDDとは質的に異なる。

第3章：VDM-SLによる仕様記述の具体例

3.1 例題：ユーザー管理システム

本章ではVDM-SLの具体例を提示する。本稿の主張は「人間が形式仕様を読む必要はない」というものだが、AIが内部で生成する成果物がどの程度厳密で、どのような保証を与えるのかを読者が実感するために、あえて具体例を示す。建築の施主が構造計算の詳細を読めなくても、構造計算書の存在と役割を知っていることに意味があるのと同じだ。

以下は、実際のWebアプリケーション開発を想定したユーザー管理システムの仕様である。

```
-- ユーザー管理システムの形式仕様

types
  UserId = nat1
  inv uid == uid <= 999999;

  Email = seq1 of char
  inv email ==
    exists i in set inds email & email(i) = '@'
    and len email <= 254;

  Password = seq of char
  inv pw == len pw >= 8 and len pw <= 128;

  Role = <Admin> | <Editor> | <Viewer>;

  User :: id       : UserId
          email    : Email
          name     : seq1 of char
          role     : Role
          active   : bool
  inv u == len u.name <= 100;

state UserSystem of
  users    : map UserId to User
  nextId   : UserId
  emails   : set of Email
inv mk_UserSystem(users, nextId, emails) ==
  -- 全ユーザーのIDはnextId未満
  (forall uid in set dom users & uid < nextId)
  -- emailsは全ユーザーのメールアドレスの集合と一致
  and emails = {users(uid).email | uid in set dom users}
  -- 各ユーザーのidフィールドはマップのキーと一致
  and (forall uid in set dom users & users(uid).id = uid)
init s == s = mk_UserSystem({|->}, 1, {})
end
```

```
operations

  RegisterUser(email: Email, name: seq1 of char, role: Role) uid: UserId
    ext wr users  : map UserId to User
        wr nextId : UserId
        wr emails : set of Email
    pre email not in set emails   -- メールアドレスの一意性
        and nextId <= 999999      -- IDの上限
    post let newUser = mk_User(nextId~, email, name, role, true) in
         uid = nextId~
         and users = users~ munion {nextId~ |-> newUser}
         and nextId = nextId~ + 1
         and emails = emails~ union {email};

  DeactivateUser(uid: UserId)
    ext wr users : map UserId to User
    pre uid in set dom users
        and users(uid).active = true
    post users = users~ ++
         {uid |-> mu(users~(uid), active |-> false)};

  ChangeRole(uid: UserId, newRole: Role)
    ext wr users : map UserId to User
    pre uid in set dom users
        and users(uid).active = true
    post users = users~ ++
         {uid |-> mu(users~(uid), role |-> newRole)};

  FindUserByEmail(email: Email) result: [UserId]
    ext rd users : map UserId to User
    post if exists uid in set dom users & users(uid).email = email
         then result <> nil
              and result in set dom users
              and users(result).email = email
         else result = nil;
```

```
functions

  ActiveUserCount: map UserId to User -> nat
  ActiveUserCount(users) ==
    card {uid | uid in set dom users & users(uid).active};

  HasAdmin: map UserId to User -> bool
  HasAdmin(users) ==
    exists uid in set dom users &
      users(uid).role = <Admin> and users(uid).active;
```

3.2 この仕様が直接保証していること

上記の仕様を注意深く読むと、以下の性質が論理的に保証されていることがわかる。

1. メールアドレスの一意性： `RegisterUser` の事前条件 `email not in set emails` により、同一メールアドレスでの二重登録は仕様レベルで不可能である。TDDでこれを保証するには、あらゆる登録順序・並行実行パターンでの重複テストが必要になるが、網羅は不可能だ。

2. 状態の一貫性： 不変条件 `inv mk_UserSystem(...)` により、`emails` 集合と `users` マップの同期、IDの整合性が常に保たれる。操作の事後条件がこの不変条件を維持することは、仕様から演繹的に確認できる。

3. 型の制約： `UserId` は1以上999999以下の自然数、`Email` は `@` を含む1文字以上254文字以下の文字列、`Password` は8文字以上128文字以下——これらの制約が型定義に組み込まれている。

3.3 仕様の「契約」が生み出す設計課題

3.2節は仕様が直接保証する性質だった。しかし、形式仕様のもう一つの重要な機能は、仕様そのものでは解決しないが、仕様が明示的であるからこそ設計の議題として浮上する問題群を生み出すことだ。

事前条件は責任境界の契約である。 たとえば `ChangeRole` の事前条件 `users(uid).active = true` は、「非アクティブユーザーのロール変更は、この操作の責任範囲外である」と宣言している。不正な入力が物理的に消滅するわけではない——「この条件が成り立たない場合、この操作の振る舞いは保証しない」という契約だ。

この明示的な契約は、2つの設計課題を具体化する。

設計課題**1**：モジュール間の合成可能性の検証。　モジュールAが `RegisterUser` でユーザーを作成し（事後条件：`active = true`）、直後にモジュールBが `ChangeRole` を呼ぶ場合、Aの事後条件がBの事前条件を含意する（A.post ⇒ B.pre）ことを形式的に確認できる。モジュールの接続が仕様レベルで矛盾しないことの証明であり、テストでは達成が困難な保証だ。この検証はPhase 2の設計段階でAIが自動的に実施する。

設計課題**2**：防御の配置。　事前条件を満たさないデータがシステムに到達したとき、どこで・どのように防御するかを決定しなければならない。バリデーション層をAPIゲートウェイに置くか、各モジュールの入口に置くか、あるいはデータの検疫（サニタイズ）モジュールをシステムのどの層に挟むか。これは仕様の問題ではなく設計の問題だが、事前条件が明示されているからこそ「何を・どこで・どう防ぐか」が具体的な設計課題として見える化される。曖昧な自然言語仕様では、この防御設計の議論自体が発生しないまま実装に進んでしまうリスクがある。

これらの設計課題は、第4章のワークフローにおけるPhase 2（AIによる設計）で具体的に解決される。形式仕様は「正しさの保証」と「設計課題の明確化」という2つの価値を同時に提供する——前者が3.2節、後者がこの3.3節の主題だ。

第**4**章：**AI**駆動開発ワークフロー

4.1 提案するワークフロー

本稿が提案する開発ワークフローは以下の通りだ。

Phase 1：施主とハウスメーカー**——**対話による形式仕様の策定

このフェーズは、建築における施主とハウスメーカーの関係に似ている。施主（人間）は「日当たりが良くて開放的なリビングが欲しい」と要望を伝える。ハウスメーカー（AI）は予算・立地・気候などの制約を考慮して「吹き抜けでガラス張りはいかがでしょうか」と提案する。施主はその提案を評価し、受け入れるか別の要望を出す。ハウスメーカーはさらに、ガラスの厚みやフレームの仕様、吹き抜けに伴う空調対策（シーリングファンの設置等）を設計し、予算に収まるかを検証する。もし寒冷地在住で空調費用を抑えたいという要件があれば、吹き抜けガラス張りという提案は撤回し、別の方法を提案するだろう。こうした思考プロセスを経て仕様を作り上げていくのがPhase 1だ。

人間がビジネス要件を自然言語で伝え、AIがVDM-SL仕様を生成する。人間は形式仕様を直接読む必要はない。AIが仕様の内容を自然言語で説明し、人間はその説明に対して判断を下す。

具体的な対話の流れはこうだ。人間が「ユーザー登録時にメールアドレスの重複を防ぎたい」と伝える。AIが「事前条件として`email not in set emails`を定義しました。これにより、既に登録済みのメールアドレスでの再登録は論理的に不可能になります。ただし、退会済みユーザーのメールアドレスの再利用については、許可しますか？」と返す。人間はドメイン知識に基づいて判断する。

このフェーズで人間が担うのは以下だ。

- ビジネスドメインの知識を提供する
- AIの説明を聞き、仕様の妥当性（validation）を判断する——「それは意図した振る舞いか？」
- エッジケースについてAIに質問する——「退会したユーザーが再登録したらどうなる？」
- 最終的な仕様の承認

AIが担うのは、自然言語のビジネス要件をVDM-SLに変換すること、仕様の内部整合性をチェックすること、仕様の内容と含意を人間に分かりやすく説明すること、そして潜在的なエッジケースや矛盾を人間に提示することだ。

重要なのは、形式仕様そのものはAIが読み書きする成果物だという点だ。人間がVDM-SLの記法を理解する必要はない。では、なぜ自然言語の仕様書ではなくVDM-SLで書くのか。この疑問については第5.3節で詳述する。

もう一つ具体例を示そう。「数値の並べ替え」という、一見単純な要件のPhase 1対話だ。

> 人間： 入力された数値を小さい順に並べ替えて出力したい。 ***AI***： *`{3, 2, 7}`* が入力されたら *`[2, 3, 7]`* を出力する、という理解でよいですか？ 人間： そう。 ***AI***： 確認ですが、*`{3, 2, 7}`* に対して *`[1, 2, 3]`* ではダメですか？　つまり「小さい順の列」であればよいのか、「入力の要素だけからなる列」でなければならないのか、という点です。 人間： 入力の要素がすべて含まれていなければならない。 ***AI***： では *`{3, 3, 1, 8}`* に対して *`[1, 3, 8]`* でしょうか？　重複する要素は*1*つにまとめてよいですか？ 人間： いや、元の数値をそのまま並べ替えるだけだ。重複もそのまま残る。

この3往復の対話を経て、AIは以下のVDM-SL仕様を生成する。

```
functions

sort[@T]: seq of @T * (@T * @T -> bool) -> seq of @T
sort(l, less) == is not yet specified
post bagOf[@T](l) = bagOf[@T](RESULT)              -- 置換条件
     and forall i in set {1, ..., len RESULT - 1} &
         not less(RESULT(i+1), RESULT(i))           -- 整列条件
measure len l;

bagOf[@T]: seq of @T -> map @T to nat
bagOf(s) ==
  { i |-> occurs[@T](i, s) | i in set elems s }
post dom RESULT = elems s and sizeOfBag[@T](RESULT) = len s;

occurs[@T]: @T * seq of @T -> nat
occurs(e, s) ==
  if s = [] then 0
  else (if e = hd s then 1 else 0) + occurs[@T](e, tl s)
measure len s;
```

注目すべきは、自然言語で3往復の誤解が生じた「並べ替え」の意味が、VDM-SLでは2つの事後条件で一意に確定している点だ。`bagOf(l) = bagOf(RESULT)` は「出力は入力の置換（同じ要素が同じ回数だけ存在する）」を、整列条件は「隣接要素が逆転しない」を意味する。この仕様はバブルソート、クイックソート、マージソートのいずれで実装しても満たされる。アルゴリズムの選択は仕様ではなく設計の問題だ。

Phase 2：**AI**による技術選定と設計——非機能要件との統合

Phase 1で確定した形式仕様は「何を計算するか（What）」を定義するが、「どの程度の規模・速度・コストで計算するか」は定義しない。Phase 2では、形式仕様とは別に人間から提示される 非機能要件を統合し、技術的な設計判断を行う。

たとえば、先のソート仕様に対して人間が「データ規模は$n = 10^{30}$程度」「計算資源と予算は限定的」「安定ソートである必要はない」と伝えたとする。この非機能要件と仕様を組み合わせて、AIは以下を決定する。

- プログラミング言語の選定（型安全性、パフォーマンス要件、エコシステムを考慮）

- アルゴリズムの選定（n = 10^{30}なら外部ソートが必須、安定性不要ならクイックソート系が候補に）
- フレームワーク・ライブラリの選定
- アーキテクチャ設計（分散処理が必要か、マイクロサービス/モノリス、データベース選定など）
- インフラ構成（サーバースペック、クラウドプロバイダ、コンテナオーケストレーション等）

重要なのは、形式仕様が「正しさの基準」を、非機能要件が「設計の制約」を、それぞれ独立に与えるという構造だ。バブルソートはVDM-SL仕様を満たすが、n = 10^{30}に対しては$O(n^2)$で実用不可能であり、非機能要件に基づいて棄却される。形式仕様と非機能要件の分離により、正しさと効率の議論が混線しない。

非機能要件の対話的引き出し： ここで実務上の問題がある。「レスポンスタイム200ms以下」のような定量的な非機能要件を、ドメインエキスパートが最初から適切に設定できるとは限らない。200msが妥当か500msが妥当かは、ユーザー体験の知見とインフラコストのトレードオフであり、純粋なドメイン知識とは異なる領域だ。

この問題に対して、本手法ではAIが体験を通じた引き出しを行うことを想定する。たとえば、AIがモックUIを生成し、意図的に異なるレスポンス遅延（100ms、300ms、500ms、1000ms）を再現して人間に操作させる。人間は「この速度では遅すぎる」「これなら許容範囲だ」と体感で判断できる。あるいは、AIがインフラコストの概算を提示し、「200msを達成するにはこの構成で月額○○円、500msならこの構成で月額○○円です」と選択肢を示すことで、ドメインエキスパートがビジネス判断として意思決定できる。非機能要件の定量化もまた、Phase 1と同様にAIが提案し人間が判断するという対話構造の中で解決される。

Phase 3：**AI**によるコード生成

VDM-SL仕様に対応するコードをAIが生成する。このフェーズでのポイントは以下だ。

- VDM-SLの事前条件は実行時バリデーション（またはアサーション）として実装される
- 事後条件はコードの正しさの検証基準となる
- 不変条件はデータ構造の設計制約となる
- 型定義は実装言語の型システムに可能な限り直接マッピングされる

ここで注目すべきは、VDM-SLの抽象型から実装の具体的なデータ構造への変換という設計判断が発生する点だ。VDM-SLは意図的に実装詳細を抽象化している。たとえば`seq of T`（順序付きコレクション）は、ランダムアクセスが頻繁なら`Array`、先頭への挿入・削除が多ければ`LinkedList`、

両方が必要なら`ArrayList`や`Deque`に変換される。同様に`set of T`は`HashSet`、`TreeSet`、`BitSet`のいずれかに、`map T1 to T2`は`HashMap`、`TreeMap`、`ConcurrentHashMap`のいずれかになり得る。

この判断は型定義だけからは決まらない。仕様中の操作の頻度パターン（検索が支配的か、挿入が支配的か）、Phase 2で確定した非機能要件（並行アクセスの有無、メモリ制約、レイテンシ要件）、そして選定済みの言語・フレームワークの特性を総合してAIが決定する。つまり、形式仕様の「何を（What）」から実装の「どのように（How）」への変換は、単純なマッピングではなく、Phase 2の設計判断と連動した最適化プロセスである。

Phase 4：仕様適合性の検証

生成されたコードが形式仕様に適合しているかを検証する。ここで初めてテストが登場するが、その役割は根本的にTDDとは異なる。テストは仕様から自動導出される性質ベースのテスト（property-based testing）であり、人間が個別のテストケースを手書きする必要はない。

仕様が厳密に定義されていることの恩恵として、テスト生成の自由度は非常に高い。事前条件を満たすランダムな入力を大量に生成し事後条件が成り立つかを検証するランダムテスト、不変条件の境界を狙った境界値テスト、状態遷移の全パスを網羅するシステマティックなテスト——これらをすべて仕様から自動生成できる。ソートの例では、`bagOf(l) = bagOf(RESULT)`と整列条件の2つの事後条件に対して、空列・単一要素・全要素同一・逆順入力・巨大列など、あらゆるパターンの入力が自動生成され検証される。

Phase 2〜4のサイクルは**AI**が自律的に回す

ここで強調すべき重要な点がある。Phase 2（設計）→ Phase 3（実装）→ Phase 4（検証）のサイクルは、すべて**AI**が自律的に実行する。Phase 4で仕様不適合が検出されれば、AIは実装を修正するか、設計判断（アルゴリズム選定やデータ構造選択）を見直して再実装する。このフィードバックループに人間は介入しない。

これが可能なのは、仕様が厳密に定義されているからだ。仕様が曖昧であれば「この不具合は仕様の問題か実装の問題か」をAIが自律的に判断することはできない。しかし、形式仕様は正しさの判定基準を一意に与えるため、AIは「仕様を満たしていない→修正が必要」という判断を自動的に下せる。人間がPhase 1で仕様を確定した後は、動くシステムが手元に届くまでAIに一任できる。

仕様中心の短サイクル——ウォーターフォールでもアジャイルでもない第三の構造

ここで本手法のサイクル構造を、従来手法と対比して明確にしておく。

ウォーターフォールでは「要件定義→仕様策定→設計→コーディング→リリース→要件定義…」と一方向に流れる。アジャイルでは「要件定義→簡易設計→コーディング→リリース→要件定義…」と、リリースを挟んだフィードバックで改善を回す。いずれも、暗黙知が顕在化するのはリリース後——つまり動くシステムを人間が使った後だ。

本手法はこの構造が根本的に異なる。形式仕様が厳密であるからこそ、リリースを待たずに仕様レベルでのテストとモック生成が可能になる。サイクルは以下の通りだ。

> 要件の言語化 ↔ 形式仕様の策定 ↔ 仕様テスト・モック提示 → リリース

まず人間が要件を伝え、AIがVDM-SL仕様を生成する。AIは仕様の内部整合性を検証し、矛盾や未定義のエッジケースを人間に提示する。さらに、仕様から自動生成されたモック（プロトタイプ）を人間に操作させることで、「使ってみて初めて気づく暗黙知」をコードを一行も書く前に引き出す。仕様のテストが論理的な矛盾を炙り出し、モックが体験的な気づきを炙り出す。この二重のフィードバックが仕様レベルで回るのだ。

建築に喩えれば、建てる前に3Dモデルで間取りを歩き回り、「この動線は使いにくい」「ここに棚が欲しい」と気づけるVR内見のようなものだ。実際に住んでからリフォームするより、はるかに効率的である。

このサイクルが十分に回った後に、Phase 2〜4をAIが自律的に実行し、仕様に適合するシステムをリリースする。もちろん、リリース後に新たな暗黙知が発見される可能性がゼロになるわけではない。しかし、仕様レベルでの論理的検証とモックによる体験的検証を経ているため、リリース後に浮上する暗黙知の量はアジャイル手法と比べて大幅に削減される。そして万一発見された場合も、同じ仕様中心サイクルに戻るだけだ。アジャイルのようにテストを書き直しリファクタリングの副作用に怯える必要はない——仕様を更新すればAIが新しいシステムを生成する。

仕様を中心としたサイクルであること——これが本手法の構造的な特徴である。

4.2 人間に求められるスキル

この開発パラダイムにおいて、人間に要求されるスキルは従来とは根本的に異なる。

必須スキル：

- ドメイン知識： 開発対象のビジネスドメインに関する深い理解。これはAIには代替できない人間固有の価値であり、最も重要なスキルである。「退会済みユーザーのメールアドレスを再利用可能にすべきか」といった判断は、ビジネスの文脈を理解する人間にしかできない。

- 論理的対話能力： AIが仕様を説明した際に、論理的な矛盾や要件の漏れを自然言語の対話を通じて指摘できる能力。形式記法を読む必要はないが、「AならばB」「すべてのXについてYが成り立つ」といった論理的構造を自然言語で理解し議論できる素養は求められる。
- 要件の言語化能力： 暗黙知として持っているビジネス要件を、AIに伝わる形で言語化する能力。これは従来の要件定義スキルと同質だが、対話相手がAIになることで、曖昧な表現を残せなくなる分、より明確な言語化が求められる。

不要になるスキル：

- 形式手法の記法の読み書き（AIが仕様を生成し、自然言語で説明する）
- 特定プログラミング言語の熟練（AIが適切な言語を選定し、コーディングする）
- フレームワークやライブラリの詳細知識（AIが最適なものを選定する）
- テストケースの設計（仕様から自動導出される）
- インフラ構成の詳細設計（仕様の非機能要件からAIが導出する）

4.3 従来の開発プロセスとの比較

観点	**TDD**	形式手法 **+ AI**駆動
正しさの保証	帰納的（有限テストケース）	演繹的（論理的証明）
仕様の曖昧性	テストが暗黙の仕様を構成	仕様が明示的・厳密
設計の駆動力	テストの書きやすさ	要件の論理構造
人間の役割	コーダー兼テスター	施主（ドメインエキスパート）
AI活用度	コード補完レベル	設計〜実装〜検証を一貫して委任
スケーラビリティ	テスト数が爆発的に増加	仕様は問題の複雑さに比例
バグ検出タイミング	実装後（テスト実行時）	仕様策定時（論理矛盾として検出）

第5章：なぜ「今」形式手法なのか

5.1 LLM以前の形式手法の課題

形式手法は1970年代から存在しながら、産業界で広く普及しなかった。その理由は主に3つあった。

第一に、形式仕様の記述には高度な数学的訓練が必要だった。VDM-SLやZ記法を書ける人材は限られており、チーム全員に習得を求めることは現実的ではなかった。

第二に、形式仕様からコードへの変換に大きなギャップがあった。美しい仕様を書いても、それを実装に落とし込む作業は結局手作業であり、ここでバグが混入した。

第三に、投資対効果が不透明だった。仕様記述にかける時間的コストに対して、得られるリターンが不明確であり、航空宇宙・鉄道・医療機器など安全性が最優先される分野に限定されていた。

5.2 LLMがゲームチェンジャーである理由

LLMの登場により、上記3つの課題がすべて解消されつつある。

課題1の解消：　人間が形式仕様を書くことも読むことも不要になった。自然言語でビジネス要件を伝え、AIがVDM-SLに変換する。AIは仕様の内容を自然言語で人間に説明し、人間はその説明に基づいて妥当性を判断する。建築の施主が構造計算書を読まずとも設計士との対話で設計を把握するのと同じ構造だ。

課題2の解消：　形式仕様からコードへの変換をAIが行う。これにより、仕様と実装のギャップが大幅に縮小する。AIは仕様の事前条件・事後条件・不変条件を理解した上でコードを生成するため、変換時のバグ混入リスクが低減する。ただし、現在のLLMは複雑な仕様の変換において幻覚（hallucination）を起こす可能性があるため、生成コードの仕様適合性検証（Phase 4）は省略できない。

課題3の解消：　AIによる自動化により、形式仕様の記述コストが劇的に下がった。従来数週間かかっていた仕様策定が、AIとの対話により数時間〜数日で完了する。投資対効果の方程式が根本的に変わったのだ。

5.3「AIだけが読むなら形式手法は不要」への反論

ここで自然に生じる疑問がある。人間が形式仕様を読まないのであれば、わざわざVDM-SLで書く必要はないのではないか。AIが自然言語で仕様を管理すれば十分ではないか。

この疑問に対する回答は明確だ。形式記法は**AI**にとっても思考の規律として機能する。

第一に、形式記法は曖昧性を構造的に排除する。自然言語で「ユーザーはメールアドレスを一つ持つ」と書いた場合、これが「正確に一つ」なのか「少なくとも一つ」なのか「最大一つ」なのかは文脈に依存する。VDM-SLで `email: Email` と型定義すれば「正確に一つ」であることが記法的に確定する。AIが自然言語で仕様を扱うと、こうした曖昧性を自ら導入してしまうリスクがある。形式記法はそのリスクを構造的に排除する。

第二に、形式仕様は機械的に検証可能な中間成果物である。VDM-SLの仕様はOverture Toolなどで型検査・整合性検査ができる。自然言語の仕様書にはこの性質がない。仕様を策定するAIと実装するAIが異なるセッションや異なるモデルであっても、形式仕様が厳密な契約として両者を拘束する。

第三に、監査可能性の問題がある。形式仕様は人間が直接読まなくても、別のAI、別のツール、あるいは将来のより高度な検証システムで事後的に検証できる。たとえば、セキュリティ監査時に「すべての操作で認証済みユーザーのみがアクセス可能か」を形式仕様上で機械的に確認できる。コンプライアンス検査時に「個人情報の削除操作が全関連テーブルに波及するか」をツールで自動証明できる。自然言語仕様に対して同様の機械的検証を行うことは、曖昧性の存在により本質的に困難だ。プロジェクトの規模が大きくなるほど、この監査可能性の価値は増大する。

つまり、形式手法は「人間のため」ではなく「論理的厳密性のため」に存在する。人間に読まれなくても、形式記法で書くこと自体にAIの推論精度を高める効果がある。これは、数学者が直感だけで考えるより記号を使って考えた方が正確な結論に至りやすいのと同じ原理だ。

5.4 AGI到来までの過渡期戦略

現在のLLMはAGI（汎用人工知能）ではない。自律的にビジネス要件を理解し、最適なシステムを設計・実装する能力は持っていない。しかし、与えられた形式仕様を理解し、それに基づいてコードを生成する能力はすでに実用レベルに達している。

この「仕様理解と実装は得意だが、要件定義は苦手」というLLMの特性は、形式手法との組み合わせにおいて理想的だ。人間がビジネスの「What（何を作るか）」を自然言語で伝え、AIがそれを形式仕様に変換して人間と対話で検証し、さらにAIが技術の「How（どう作るか）」を実装する。

AGIが到来すれば、「What」の定義すらAIが担えるようになるかもしれない。しかし、それまでの過渡期において、形式手法 + AI駆動開発は最も合理的なアプローチである。

5.5 現段階での暫定的な運用——理想と今日の橋渡し

本手法のコンセプトは「AIエージェントの自律性が進化した数ヶ月先の未来からAGI到達までの間の、AI駆動開発の最適化」である。理想的なワークフロー（Phase 2〜4の完全自律サイクル）は、現在のLLMの能力では複雑なシステムに対して完全には実現できない。この事実を誠実に認めた上で、現段階から本手法を段階的に導入するための暫定策を以下に示す。

Phase 2（設計）の暫定策： 現在のLLMは、単純なCRUDシステムやAPIサービスの設計であれば自律的に行える。しかし、複雑な分散システムや高度なセキュリティ要件を持つシステムでは、経験豊富なアーキテクトの介入が必要になる場合がある。暫定策として、AIが設計案を生成し、人間のアーキテクトがレビュー・修正するハイブリッドモデルを採用する。重要なのは、このアーキテクトの介入を過渡的なものと位置づけ、AIの自律性が向上するにつれて段階的に縮小していく前提で運用することだ。

Phase 4（検証）の暫定策： VDM-SLの事後条件からproperty-based testを自動生成し実行するサイクルは、現段階では単純な仕様に対してのみ完全に自律的に回せる。複雑な状態遷移や並行処理を含む仕様では、テスト戦略の設計にQAエンジニアの知見が必要になることがある。この場合も、QAエンジニアはテストケースを手書きするのではなく、AIが生成したテスト戦略のレビューと方針修正に徹する。テストの実行と結果判定は引き続きAIが自律的に行う。

暫定策の共通原則： いずれの暫定策においても、人間の介入は「AIの出力をレビューし、方針を修正する」という形態に限定される。人間がゼロからコードを書いたり、テストケースを手書きしたりすることはない。これにより、AIの自律性が向上した際に、人間の介入ポイントを自然に縮小できる。暫定策は本手法の理想像を否定するものではなく、理想への段階的な移行パスである。

第6章：実践に向けたロードマップ

6.1 個人レベルでの導入ステップ

Step 1：論理的な対話の練習（**1-2**週間）

まず身近なビジネスルールを論理的に言語化する練習から始める。たとえば「ECサイトの注文処理」について、「在庫がゼロの商品は注文できない」「同一ユーザーが同時に処理できる注文は1つまで」といったルールを明確な条件文として書き出す。集合・写像・全称/存在量化といった概念を自然言語レベルで理解していれば十分だ。VDM-SLの記法を学ぶ必要はない。

Step 2：**AI**との仕様策定対話を実践（**1-2**週間）

実際にAIに「ユーザー管理機能の仕様をVDM-SLで書いてほしい」と依頼し、AIが生成した仕様の説明を聞きながら、漏れや矛盾を指摘するサイクルを回す。ここでの目標は、AIの説明に対して「退会済みユーザーのデータはどうなる？」「管理者が自分自身を削除できてしまわないか？」といった質問を投げかけ、仕様を洗練できるようになることだ。

Step 3：小規模なプロジェクトで全サイクルを実践（**2-4**週間）

仕様策定からコード生成、成果物の確認までを一気通貫で行う。最初はCRUD操作を持つ小規模なAPIから始めるとよい。重要なのは、生成された成果物を実際に使ってみて、「仕様対話で言ったことと違う挙動」を見つけたらAIにフィードバックするサイクルを体験することだ。

6.2 チーム・組織レベルでの導入

組織への導入では、まず「パイロットプロジェクト」として新規の小規模サービスを1つ選び、形式手法 + AI駆動開発で構築する。既存の大規模システムをいきなり移行するのではなく、効果を実証してからスケールする。

レビュープロセスも変わる。従来のコードレビューは不要になり、「仕様対話のレビュー」が中心になる。AIとの仕様策定対話ログを共有し、ドメイン知識を持つ複数の人間が「この仕様は要件を正しく反映しているか」を議論する。コードの実装詳細はAIに委ね、人間は仕様レベルの判断に集中する。

第**7**章：限界と誠実な議論

7.1 形式手法が万能でない領域

形式手法は万能ではない。以下の領域では、補完的なアプローチが必要だ。

UI/UXの仕様化：　ユーザーインターフェースの「使いやすさ」や「美しさ」は形式化が困難であり、プロトタイピングとユーザーテストが引き続き有効だ。ただし、UIの背後にあるビジネスロジック（状態遷移、バリデーション規則など）は形式化可能である。

パフォーマンス特性：　VDM-SLは「何を計算するか」を記述するが「どれくらい速く計算するか」は記述しない。パフォーマンス要件は別途、非機能要件として扱う必要がある。

外部システムとの統合：　サードパーティAPIの実際の振る舞いは形式仕様では完全に捉えられない。境界部分ではインテグレーションテストが依然として有効だ。

7.2 テストが完全に不要になるわけではない

本稿のタイトルは挑発的だが、テストそのものを全否定しているわけではない。否定しているのは「TDDというパラダイム」——テストを設計と品質保証の中心に据えるという思想——である。

形式仕様 + AI駆動開発においても、テストは以下の役割で存続する。

- 仕様から自動生成されるプロパティベーステスト（仕様適合性の確認）
- 外部システム統合部分のインテグレーションテスト
- パフォーマンス・負荷テスト
- UIのE2Eテスト（受容テスト）

重要なのは、テストの位置づけが「中心」から「補完」に変わるということだ。

7.3 マルチエージェント協調と適用可能規模——形式仕様が拓くスケーリング

単一のAIエージェントが大規模システム全体を自律的に構築できるか、と問えば、答えは否だ。コンテキストウィンドウの制約から、数万行規模のシステム全体を一度に把握することはできない。しかし、これは問いの立て方が間違っている。人間の開発チームにおいても、システム全体を一人のエンジニアが把握しているわけではない。チーム開発が機能するのは、モジュール間のインターフェースが合意されているからだ。

形式仕様は、まさにこの「チーム開発を可能にするインターフェース合意」を厳密に提供する。そして、これこそが現在のAIエージェントでも中規模〜大規模システムの自律構築を可能にする鍵である。

形式仕様を契約としたマルチエージェントアーキテクチャ

構造は以下の通りだ。

Phase 1（人間 + アーキテクトAI）では、システム全体の形式仕様を策定する。ここでの核心はモジュール分割とモジュール間インターフェースの仕様確定——各モジュールが公開する操作の事前条件・事後条件の定義——だ。この仕様群が、すべてのエージェントにとっての「契約書」になる。

Phase 2〜4では、モジュールごとに独立したAIエージェントが並行して設計・実装・検証を行う。エージェントAは在庫管理モジュール、エージェントBは注文処理モジュール、エージェントCは決済モジュールを担当する。各エージェントのコンテキストに必要なのは「自モジュールの仕様」と「依存先モジュールのインターフェース仕様（事前/事後条件のみ）」だけであり、他モジュールの実装詳細は一切不要だ。これなら現在のコンテキストウィンドウで十分に収まる。

統合検証では、専用のインテグレーションエージェントが各モジュールの公開インターフェース仕様を突き合わせ、A.post ⇒ B.pre の合成可能性を機械的に検証する。このエージェントは実装コードを見る必要がなく、仕様レベルの整合性検証に特化できる。

この構造が自然言語仕様では不可能で、形式仕様だからこそ可能な理由は3つある。第一に、各エージェントが独立に「自分の実装が仕様を満たしているか」を判定できる。判定基準が曖昧でないため、他のエージェントに「すり合わせ」を求める必要がない。第二に、モジュール間の整合性検証が機械的に行える。事後条件と事前条件の含意関係はツールで自動検証可能だ。第三に、各エージェントが持つべきコンテキストが最小化される。他モジュールの実装を知る必要がなく、インターフェース仕様だけを参照すればよい。

これは、人間のチーム開発でAPIドキュメントやインターフェース定義（OpenAPI、Protocol Buffers等）がチームの分業を可能にしているのと同じ原理だ。ただし、自然言語のAPIドキュメントでは「正常系のレスポンスのみ記述され、異常系が未定義」といった曖昧さが残る。形式仕様はこの曖昧さを構造的に排除する。

残る課題と人間の役割

ただし、現時点で以下の課題は残る。横断的関心事（認証・認可、ロギング、エラーハンドリングパターン）の統一、共有データモデル（データベーススキーマ）の一貫性管理、そしてエージェント間のオーケストレーション（実行順序と依存関係の制御）だ。これらは原理的に不可能な問題ではなくエンジニアリングの課題であり、解決は時間の問題だ。

現時点では、これらの横断的な設計判断はPhase 2でAIが自律的に確定し、各エージェントへの指示に含めることで対処できる。人間の役割はあくまで「何を作るか」を決めるドメインエキスパートだ。建築に喩えれば、施主は「日当たりの良い開放的なリビングが欲しい」「寒い地域なので光熱費は抑えたい」という要望を伝える。基礎工事、躯体、電気、配管をどの順序で、どの業者に発注するかは施主の関心事ではなく、ハウスメーカーが予算・立地・気候を踏まえて最適な施工計画を組む。ソフトウェア開発においても同様に、技術選定（言語・フレームワーク・データベース）やアーキテクチャ設計はAIが自律的に行う領域である。

スケーリングの展望

マルチエージェント協調と形式仕様の組み合わせにより、現在のAIモデルでも中規模システム（数万行規模）への適用は理論的に可能であり、オーケストレーション層の整備が実用化の鍵となる。さらに、コンテキストウィンドウの拡大、エージェント間協調プロトコルの標準化、長期記憶の改善といった技術進化が進めば、大規模システム（十数万行以上）への適用も視野に入る。重要なのは、これらの技術進化のすべてにおいて、形式仕様がエージェント間の「厳密な契約」として機能する点だ。自然言語仕様ではエージェント数の増加に伴い協調の曖昧性が指数的に増大するが、形式仕様ではモジュール境界の整合性が常に機械的に検証可能であり、エージェント数に対してスケーラブルだ。

おわりに：**AI**エージェント駆動開発における人間の役割

マルチエージェント協調による形式手法 + AI駆動開発のパラダイムは、ソフトウェア開発における人間の役割を根本的に再定義する。これまでの開発者は「コードを書く人」だった。これからの人間は、ドメインエキスパート——「何を作るかを定義し、なぜ作るのかを伝え、AIの提案がビジネス意図に合致するかを評価する人」——になる。

これは人間の価値の低下ではなく、抽象度の上昇だ。アセンブリ言語から高級言語への移行、手動メモリ管理からGCへの移行と同じ進化の延長線上にある。人間はより本質的な問題——「何を作るべきか」「なぜ作るのか」——に集中できるようになる。技術選定やシステムアーキテクチャの設計はAIに委ねられる。

必要なのは、自分のビジネスドメインに対する深い理解とAIと論理的に対話する能力だ。形式記法を読み書きする必要はない。それはAIエージェントが担う。人間は、アーキテクトAIが「この仕様でモジュール分割は適切ですか？」と自然言語で説明した内容に対して、ドメインエキスパートとして判断を下す。システムがPythonで書かれるかRubyで書かれるかTypeScriptで書かれるかは人間の関心事ではなく、AIが仕様と非機能要件に基づいて自律的に判断する。そのドメイン判断の質こそが、マルチエージェント開発時代における人間のコアコンピタンスになる。

形式手法は、個々のAIエージェントの思考の規律として、エージェント間の厳密な契約として、モジュール境界の機械的検証手段として、そして将来の監査のための記録として機能する。人間が読まなくても、形式的に書くこと自体に——特に複数のエージェントが協調する文脈において——決定的な価値がある。これが本稿の中心的な主張だ。

TDDが支配した時代は終わりを告げつつある。形式手法をエージェント間の「契約」として活用する、マルチエージェントAI駆動開発の夜明けに、我々は立ち会っている。そして、この新しいパラダイムにおいて人間が果たすべき役割は、コードを書くことでも技術を選定することでもなく、AIエージェントチームが正しいものを作るためのドメイン知識の提供と要件の意思決定にある。

参考文献

- Dijkstra, E.W. (1969). *Notes on Structured Programming*.
- Jones, C.B. (1990). *Systematic Software Development using VDM*. Prentice Hall. — VDMの基本書。
- Fitzgerald, J. & Larsen, P.G. (2009). *Modelling Systems: Practical Tools and Techniques in Software Development*. Cambridge University Press. — VDM-SLの実践的解説。
- Lamport, L. (2002). *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley.
- Newcombe, C. et al. (2015). “How Amazon Web Services Uses Formal Methods.” *Communications of the ACM*, 58(4). — AWSでのTLA+適用事例。
- Jackson, D. (2012). *Software Abstractions: Logic, Language, and Analysis*. MIT Press. — Alloyの入門書。
- Overture Tool Project: https://www.overturetool.org/ — VDM-SLのオープンソースツール環境。
- Larsen, P.G. et al. (2010). “Industrial Applications of VDM.” In *Bentley Historical Library*. — VDMの産業適用事例集。
- Bicarregui, J. et al. (2009). “Proof and Model Checking for Protocol Design.” *Formal Aspects of Computing*, 21(1-2). — 形式手法の産業適用に関する包括的議論。